EXPERIMENT 4

QUESTION: Bayesian Classification

Part 1: Single Value, Multiple Class
Implement Bayesian classification where one input value is used to determine which class it belongs to among multiple classes.
Part 2: Multiple Value, Multiple Class
Implement Bayesian classification where multiple input values/features are used to determine which class the data belongs to among multiple classes.

THEORY

This implementation builds a Discrete (Categorical) Naive Bayes Classifier from scratch.

To handle categorical data (like text or labels) instead of numbers, this algorithm counts frequencies and applies Laplace Smoothing (adding 1 to all counts). This ensures that if the algorithm encounters a combination it has never seen before, it assigns a very small probability rather than crashing with a probability of absolute zero.

The posterior probability for a class is calculated using Bayes' Theorem (ignoring the denominator/evidence since it is constant across all classes):

$$P(c|X) \propto P(c) \prod_{i=1}^{n} P\,(x_i|c)$$

The above formula is for multiple input features. For single feature, the formula is

$$P(c|x) \propto P(c)P(\mathrm{x}|c)$$

CODE

```
from flask import Flask, request, render_template_string

app = Flask(__name__)

# ==========================================
# 1. Discrete Naive Bayes Engine (From Scratch)
# ==========================================
class DiscreteNaiveBayes:
    def __init__(self):
        self.classes = []
        self.priors = {}
        self.likelihoods = {}

    def fit(self, X, y):
        self.classes = list(set(y))
        total_samples = len(y)

        # Calculate Prior Probabilities P(Class)
        self.priors = {c: y.count(c) / total_samples for c in self.classes}

        num_features = len(X[0])
```

```
        # Calculate Likelihoods P(Feature | Class) with Laplace Smoothing
        for feature_idx in range(num_features):
            self.likelihoods[feature_idx] = {}
            unique_values = set(row[feature_idx] for row in X)
            num_unique = len(unique_values)

            for c in self.classes:
                self.likelihoods[feature_idx][c] = {}

                # Get all feature values where the target class matches 'c'
                feature_vals_for_class = [X[i][feature_idx] for i in range(total_samples) if y[i] == c]
                class_count = len(feature_vals_for_class)

                for val in unique_values:
                    val_count = feature_vals_for_class.count(val)
                    # Laplace Smoothing: (count + 1) / (class_count + num_unique)
                    smoothed_prob = (val_count + 1) / (class_count + num_unique)
                    self.likelihoods[feature_idx][c][val] = smoothed_prob

    def predict_proba(self, x_input):
        posteriors = {}

        for c in self.classes:
            # Start with the prior probability P(C)
            prob = self.priors[c]

            # Multiply by the likelihood of each discrete feature P(x_i | C)
            for feature_idx, val in enumerate(x_input):
                # If a value was never seen in training, assign a tiny probability
                likelihood = self.likelihoods[feature_idx][c].get(val, 1e-6)
                prob *= likelihood

            posteriors[c] = prob

        # Normalize to percentages
        total_prob = sum(posteriors.values())
        return {c: (p / total_prob) * 100 for c, p in posteriors.items()}

# ========================================
# 2. Datasets & Model Training
# Application: Predicting Transportation Mode
# ========================================

# PART 1: Single Discrete Feature -> 3 Classes
# Feature: Weather | Target: Mode of Transport
X_single = [['Sunny'], ['Sunny'], ['Rain'], ['Rain'], ['Overcast'], ['Overcast'], ['Sunny'], ['Rain'],
['Overcast']]
y_single = ['Bike', 'Car', 'Bus', 'Car', 'Bike', 'Bus', 'Bike', 'Bus', 'Car']

model_part1 = DiscreteNaiveBayes()
model_part1.fit(X_single, y_single)
```

```
# PART 2: Multiple Discrete Features -> 3 Classes
# Features: [Weather, Distance, Traffic] | Target: Mode of Transport
X_multi = [
    ['Sunny', 'Short', 'Low'],
    ['Sunny', 'Long', 'High'],
    ['Rain', 'Short', 'High'],
    ['Rain', 'Long', 'Low'],
    ['Overcast', 'Short', 'Low'],
    ['Overcast', 'Long', 'High'],
    ['Sunny', 'Medium', 'Low'],
    ['Rain', 'Medium', 'High'],
    ['Overcast', 'Medium', 'Low']
]
y_multi = ['Bike', 'Car', 'Bus', 'Bus', 'Bike', 'Car', 'Bike', 'Bus', 'Car']

model_part2 = DiscreteNaiveBayes()
model_part2.fit(X_multi, y_multi)

# ========================================
# 3. Flask HTML Template & Routing
# ========================================
HTML_TEMPLATE = """
<!DOCTYPE html>
<html>
<head>
    <title>Discrete Bayesian Classification</title>
    <style>
        body { font-family: Arial, sans-serif; background: #f4f7f6; margin: 0; padding: 30px; }
        .container { max-width: 900px; margin: auto; }
        .card { background: white; padding: 25px; border-radius: 10px; box-shadow: 0 5px 15px
rgba(0,0,0,0.08); margin-bottom: 30px; border-left: 5px solid #8e44ad; }
        h1, h2 { color: #2c3e50; }
        .result-box { background: #fdfefe; padding: 15px; border-radius: 8px; margin-top: 15px; border:
1px solid #dcdcdc; }
        .winner { font-size: 1.3em; font-weight: bold; color: #27ae60; }
        select { padding: 8px; width: 150px; border: 1px solid #ccc; border-radius: 4px; margin-right:
15px; }
        button { background: #8e44ad; color: white; border: none; padding: 10px 20px; cursor: pointer;
border-radius: 5px; font-weight: bold; }
        button:hover { background: #9b59b6; }
        table { width: 100%; border-collapse: collapse; margin-top: 15px; }
        th, td { border: 1px solid #ddd; padding: 10px; text-align: left; }
        th { background: #f9f9f9; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Experiment 4: Discrete Bayesian Classification</h1>

        <!-- PART 1: Single Feature -->
```

```
    <div class="card">
      <h2>Part 1: Single Value, Multiple Class</h2>
      <p><strong>Scenario:</strong> Predict transportation mode based ONLY on Weather.</p>
      <form method="POST">
        <input type="hidden" name="form_type" value="part1">
        <label>Weather: </label>
        <select name="weather1">
          <option value="Sunny">Sunny</option>
          <option value="Overcast">Overcast</option>
          <option value="Rain">Rain</option>
        </select>
        <button type="submit">Classify</button>
      </form>

      {% if result1 %}
      <div class="result-box">
        <h3>Classification Results</h3>
        <table>
          <tr><th>Class (Mode)</th><th>Posterior Probability P(C|X)</th></tr>
          {% for cls, prob in result1.items() %}
          <tr>
            <td>{{ cls }}</td>
            <td>{{ "%.2f"|format(prob) }}%</td>
          </tr>
          {% endfor %}
        </table>
        <p>Selected Mode: <span class="winner">{{ winner1 }}</span></p>
      </div>
      {% endif %}
    </div>

    <!-- PART 2: Multiple Features -->
    <div class="card" style="border-left-color: #2980b9;">
      <h2>Part 2: Multiple Value, Multiple Class</h2>
      <p><strong>Scenario:</strong> Predict transportation mode based on Weather, Distance, and
Traffic.</p>
      <form method="POST">
        <input type="hidden" name="form_type" value="part2">

        <label>Weather: </label>
        <select name="weather2">
          <option value="Sunny">Sunny</option>
          <option value="Overcast">Overcast</option>
          <option value="Rain">Rain</option>
        </select>

        <label>Distance: </label>
        <select name="distance">
          <option value="Short">Short</option>
          <option value="Medium">Medium</option>
          <option value="Long">Long</option>
```

```
            </select>

            <label>Traffic: </label>
            <select name="traffic">
              <option value="Low">Low</option>
              <option value="High">High</option>
            </select>

            <button type="submit" style="background: #2980b9;">Classify</button>
          </form>

          {% if result2 %}
          <div class="result-box">
            <h3>Classification Results</h3>
            <table>
              <tr><th>Class (Mode)</th><th>Posterior Probability P(C|X)</th></tr>
              {% for cls, prob in result2.items() %}
              <tr>
                <td>{{ cls }}</td>
                <td>{{ "%.2f"|format(prob) }}%</td>
              </tr>
              {% endfor %}
            </table>
            <p>Selected Mode: <span class="winner">{{ winner2 }}</span></p>
          </div>
          {% endif %}
        </div>
      </div>
    </body>
    </html>
    """

    @app.route('/', methods=['GET', 'POST'])
    def index():
      result1, winner1 = None, None
      result2, winner2 = None, None

      if request.method == 'POST':
        form_type = request.form.get('form_type')

        if form_type == 'part1':
          x_input = [request.form.get('weather1')]
          probs = model_part1.predict_proba(x_input)
          result1 = dict(sorted(probs.items(), key=lambda item: item[1], reverse=True))
          winner1 = max(probs, key=probs.get)

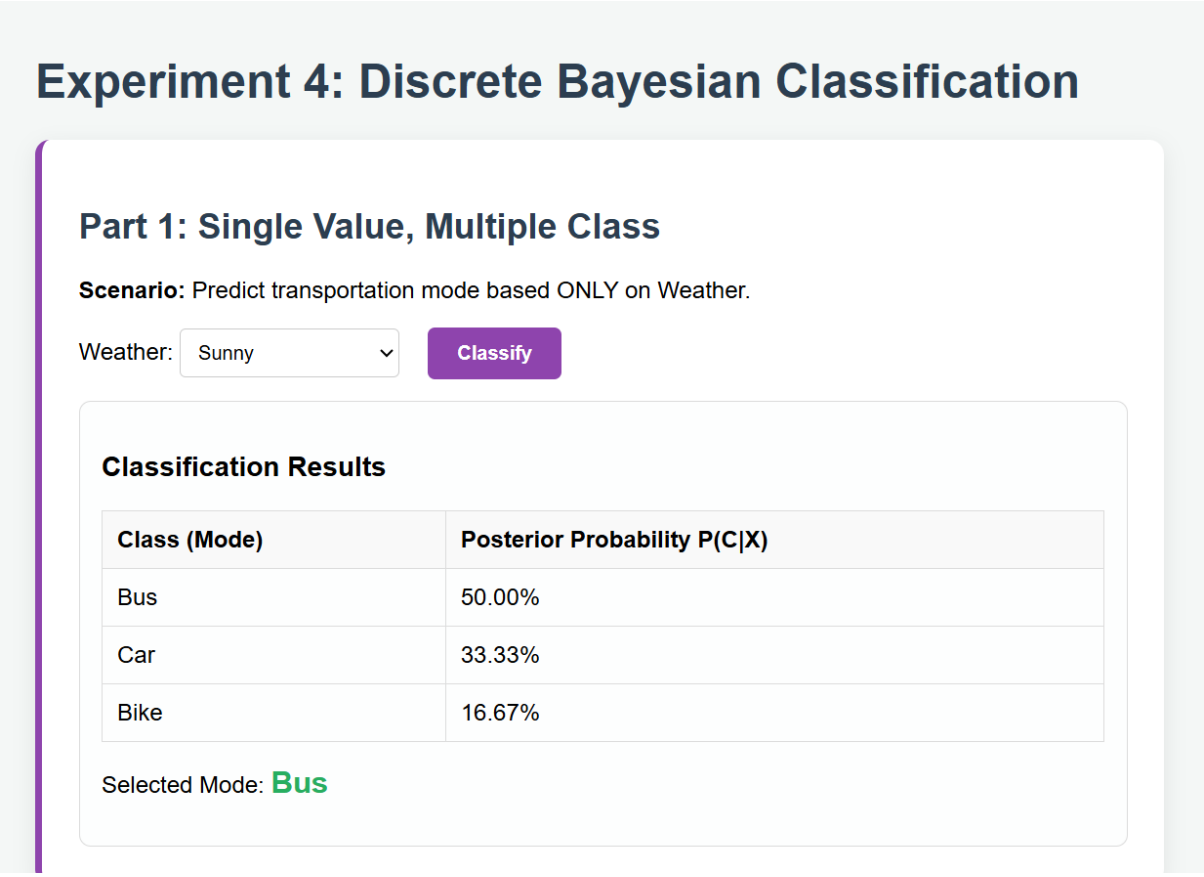
        elif form_type == 'part2':
          x_input = [
            request.form.get('weather2'),
            request.form.get('distance'),
            request.form.get('traffic')
```

```
        ]
        probs = model_part2.predict_proba(x_input)
        result2 = dict(sorted(probs.items(), key=lambda item: item[1], reverse=True))
        winner2 = max(probs, key=probs.get)

    return render_template_string(HTML_TEMPLATE,
                                  result1=result1, winner1=winner1,
                                  result2=result2, winner2=winner2)

if __name__ == '__main__':
    print("Starting Discrete Bayesian Classifier App...")
    print("Access at: http://127.0.0.1:5000")
    app.run(debug=True)
```

OUTPUT



Part 2: Multiple Value, Multiple Class

Scenario: Predict transportation mode based on Weather, Distance, and Traffic.

Weather: Sunny Distance: Short Traffic: Low

Classify

Classification Results

Class (Mode)	Posterior Probability P(C|X)
Bike	50.00%
Car	37.50%
Bus	12.50%

Selected Mode: **Bike**